

贪吃蛇项目

二、数据结构

- 知识点:自定义单向链表
 - 干什么? — 动态管理一组物理内存不连续的节点
 - 实现功能? — 实现蛇身体的动态伸缩
 - 优缺点?
 - 优: 头部插入极快 $O(1)$
 - 缺: 尾部删除需从头遍历
 - 与其他区别? — 区别于数组/vector: 无需连续内存, 不会因内存空间不足引发整体数据搬迁卡顿
 - 常犯错误? — 析构函数未遍历 delete 所有节点, 导致严重内存泄漏; 删尾未置空导致野指针
 - 注意事项? — 每次操作 next 指针前, 务必判定当前指针是否为 nullptr 防止段错误
 - 是否可优化? — 可升级为带尾指针的“双向链表”

三、算法与数学逻辑

- 知识点:网格对齐与坐标离散化算法
 - 干什么? — 将连续的像素坐标映射并限制为固定步长的网格坐标
 - 实现功能? — 确保蛇头移动步幅与苹果生成的坐标能完美重合, 从而触发“吃掉”判定
 - 优缺点?
 - 优: 数学逻辑简单, 碰撞检测性能极高
 - 缺: 蛇只能一格格跳跃, 无法顺滑微调角度
 - 与其他区别? — 区别于像素级物理碰撞: 只需判定坐标是否全等
 - 常犯错误? — 随机化坐标时忘了减去边框墙壁厚度, 导致苹果生成在墙壁区域内部
 - 注意事项? — 公式必须是 $(\text{rand}() \% \text{cols}) * \text{size}$
 - 是否可优化? — 配合插值动画 (Interpolation), 让数据底层走格子, 但在屏幕渲染上实现平滑移动;
- 知识点:AABB 包围盒与边界碰撞检测
 - 干什么? — 对蛇头的极限坐标进行边界越界判定
 - 实现功能? — 实现蛇“撞墙死亡”以及“咬到自己身体死亡”的核心游戏规则
 - 优缺点?
 - 优: 运算开销极小, 仅涉及四则比较运算
 - 缺: 针对复杂不规则多边形判定无力
 - 与其他区别? — 区别于复杂的射线检测或多边形检测, AABB 仅适用于纯粹的轴对齐矩形
 - 常犯错误? — 判定坐标极限值时, 写成了 < 0 , 忽略了自带的 30 像素边框厚度
 - 注意事项? — 由于蛇头和墙壁都占据面积, 坐标判定时一定要加上墙壁的 offset 偏移量
 - 是否可优化? — 检测自撞时, 可跳过蛇身的前几个节点, 减少不必要的循环遍历

四、游戏引擎核心机制

- 知识点:固定时间步长主循环
 - 干什么? — 解耦“按键捕获频率”与“游戏逻辑更新频率”
 - 实现功能? — 保证蛇以恒定速度移动, 同时按键能被毫秒级瞬间响应
 - 优缺点?
 - 优: 游戏体验顺畅, 极速响应不卡键
 - 缺: 需手动维护系统时间差变量
 - 与其他区别? — 区别于使用 Sleep(150) 阻塞等待: 阻塞会挂起线程, 导致休眠期间的键盘输入全部丢失
 - 常犯错误? — 在主循环内错误使用了 getch() 等阻塞型输入函数导致画面彻底卡死
 - 注意事项? — 需调用 GetTickCount() 获取系统毫秒级时间戳, 计算差值后才放行 Update()
 - 是否可优化? — 可引入 Delta Time 机制分离逻辑帧与渲染帧, 适应任意高刷新率显示器
- 知识点:死亡定格与时序阻断
 - 干什么? — 精确控制 Update() 状态机与 Render() 渲染层的执行时机
 - 实现功能? — 完美解决撞墙后, 蛇头依然会渲染穿透进墙壁 1 帧的严重视觉 Bug
 - 优缺点?
 - 极其简单的逻辑修复了致命表现缺陷
 - 基于硬编码控制流程, 略失优雅
 - 与其他区别? — 区别于常规的顺序瀑布流执行: 在更新后立刻插入断点拦截, 不让错误状态流入渲染层
 - 常犯错误? — 撞墙判定 GameOver = true 后, 放任程序继续走完当前循环的 Render()
 - 注意事项? — 时序极度敏感, break 拦截代码必须紧紧跟随在 Update() 之后、Render() 之前
 - 是否可优化? — 可引入成熟的状态机 (State Machine) 模式, 死后切换至 DEAD_STATE 交由专门的定格逻辑渲染

五、EasyX 图形渲染

- 知识点:双缓冲绘图
 - 干什么? — 在显存后台完成所有元素的绘制, 再整体交换至前台屏幕
 - 实现功能? — 彻底终结清屏和挨个画图带来的画面剧烈闪烁现象
 - 优缺点?
 - 优: 画面呈现极度平滑无割裂感
 - 缺: 占用了双倍的显存缓冲区内存
 - 与其他区别? — 区别于单缓冲: 单缓冲在绘制多个元素时存在时间差, 肉眼会捕捉到残影闪烁
 - 常犯错误? — 漏写 EndBatchDraw(), 导致程序后台运行正常但屏幕全黑毫无画面
 - 注意事项? — BeginBatchDraw 和 EndBatchDraw 必须严格成对存在, 包裹所有的图形化输出语句
 - 是否可优化? — 可进阶为“脏矩形渲染”(Dirty Rectangles), 每帧只重绘苹果和蛇变化的网格区, 极致压缩显卡开销
- 知识点:图像资源加载与映射
 - 干什么? — 将外部硬盘的 .bmp 等图片文件载入内存并渲染到游戏界面
 - 实现功能? — 使游戏摆脱单调粗糙的几何线条, 具备真实美观的美术质感
 - 优缺点?
 - 优: 视觉效果拔群
 - 缺: 需考虑路径寻找、透明通道融合等额外问题
 - 与其他区别? — 区别于画方块: 使用素材意味着对象具备了材质属性
 - 常犯错误? — 文件相对路径写错 (漏掉文件夹目录), 导致加载失败, 蛇直接变成“隐身状态”
 - 注意事项? — loadimage 载入时, 务必在参数中强制设定宽和高为网格 size, 防止原生图片尺寸导致贴图错乱越界
 - 是否可优化? — 可引入贝叶斯 Alpha 混合公式transparentimage 函数, 去除蛇头图片丑陋的方块底色, 实现完美镂空

一、工程架构与面向对象思想

- 知识点:声明与实现分离
 - 干什么? — 将类的接口声明与具体的实现代码物理隔离
 - 实现功能? — 实现高内聚、低耦合的模块化设计, 使代码结构清晰
 - 优缺点?
 - 优点: 提升编译速度,利于多人协作
 - 缺点: 维护两套代码,开发时需要频繁切换
 - 与其他区别? — 全部写在单个文件里,代码可读性差,代码臃肿, 难以定位bug
 - 常犯错误? — 忘记include头文件
 - 注意事项? — 必须要include头文件
 - 是否可优化? — ?
- 知识点:头文件防卫式声明#pragma once
 - 干什么? — 指示编译器在编译该工程时, 只包含一次该头文件
 - 实现功能? — 彻底解决头文件被相互、循环包含导致的“类型重定义”崩溃
 - 优缺点?
 - 优: 写法极其简便, 编译解析速度快
 - 缺: 部分极古老编译器可能不支持
 - 与其他区别? — 区别于传统的 #ifndef...#define 宏定义防卫, 不容易发生宏名冲突
 - 常犯错误? — 在新建类的 .h 文件时忘记写在第一行
 - 注意事项? — 它是现代 C++ 工程的标配, 写任何头文件先敲此代码
 - 是否可优化? — 当前最优